7. HotStuff 모듈 시스템

7.1 모듈 시스템 개요

7.1.1 모듈 시스템의 필요성

HotStuff 구현에서 모듈 시스템이 필요한 이유:

- **순환 의존성**: 모듈들이 서로 의존하는 복잡한 관계
- **구성 문제**: 다양한 모듈 조합으로 시스템 구성
- **확장성**: 새로운 모듈 구현의 용이성
- **테스트**: 모듈별 독립적 테스트 가능

7.1.2 설계 원칙

```
┌─────────────────┐ ┌─────────────────┐
│      의존성 주입    │ │    지연 초기화    │
│   (Dependency   │ │   (Deferred     │
│     Injection)  │ │  Initialization)│
└─────────────────┘ └─────────────────┘
        │                     │
        └─────────┬───────────┘
                  │
          ┌─────────────────┐
```

```
|  모듈 레지스트리  |
|(Module Registry)|
└──────────────────┘
```

7.2 핵심 인터페이스

7.2.1 Module 인터페이스

```go
type Module interface {
InitModule(mods *modules.Core)
}
```

역할:

- 모듈 초기화 시점에 호출됨
- 필요한 의존성을 `mods` 에서 가져옴
- 이벤트 핸들러 등록 가능

7.2.2 Provider 인터페이스 (선택적)

```go
type Provider interface {
ModuleType() any
}

// 제네릭을 사용한 편의 구조체
```

```go
type Implements[T any] struct{}


func (Implements[T]) ModuleType() any {
return  new(T)
}
```

사용 예:

```go
type MyConsensus struct {
modules.Implements[modules.Consensus]
// ... 필드들
}
```

7.3 Core 모듈 관리자

7.3.1 Core 구조체

```go
type Core struct {
modules  map[reflect.Type]any
mutex sync.RWMutex
// 설정
opts *Options
// 이벤트 루프
eventLoop *eventloop.EventLoop
```

```
}
```

7.3.2 의존성 주입 메서드

```
// 단일 모듈 가져오기
func (c *Core) Get(ptrs ...any) {
c.mutex.RLock()
defer  c.mutex.RUnlock()
for  _, ptr := range  ptrs {
c.getModule(ptr)
}
}

// 모듈 가져오기 (실패 시 패닉 없음)
func (c *Core) TryGet(ptrs ...any) {
c.mutex.RLock()
defer  c.mutex.RUnlock()
for  _, ptr := range  ptrs {
c.tryGetModule(ptr)
}
}

// 모든 모듈 가져오기
```

```go
func (c *Core) GetAll(ptrs ...any) {
c.Get(ptrs...)
}
```

7.3.3 모듈 검색 로직

```go
func (c *Core) getModule(ptr any) {
ptrVal := reflect.ValueOf(ptr)
if ptrVal.Kind() != reflect.Ptr {
panic("argument must be a pointer")
}
elemType := ptrVal.Elem().Type()
// 정확한 타입 매치 시도
if module, exists := c.modules[elemType]; exists {
ptrVal.Elem().Set(reflect.ValueOf(module))
return
}
// 인터페이스 호환성 확인
for moduleType, module := range c.modules {
if reflect.TypeOf(module).AssignableTo(elemType) {
ptrVal.Elem().Set(reflect.ValueOf(module))
return
}
```

```
}
panic(fmt.Sprintf("module of type %v not found", elemType))
}
```

7.4 Builder 패턴

7.4.1 Builder 구조체

```
type Builder struct {
modules []any
options []Option
}

func  NewBuilder() *Builder {
return &Builder{
modules: make([]any, 0),
options: make([]Option, 0),
}
}
```

7.4.2 모듈 추가

```
func (b *Builder) Add(modules ...any) *Builder {
b.modules = append(b.modules, modules...)
```

```go
return b
}

func (b *Builder) AddOption(opts ...Option) *Builder {
b.options = append(b.options, opts...)
return b
}
```

7.4.3 시스템 구축

```go
func (b *Builder) Build() *Core {
core := &Core{
modules: make(map[reflect.Type]any),
opts: NewOptions(),
}
// 옵션 적용
for _, opt := range b.options {
opt(core.opts)
}
// 이벤트 루프 생성
core.eventLoop = eventloop.New(core.opts.EventLoopBufferSize())
// 모듈 등록
for _, module := range b.modules {
```

```
core.registerModule(module)
}
// 모듈 초기화
core.initModules()
return core
}
```

7.4.4 모듈 등록 과정

```
func (c *Core) registerModule(module any) {
moduleType := reflect.TypeOf(module)
// Provider 인터페이스 확인
if provider, ok := module.(Provider); ok {
providedType := reflect.TypeOf(provider.ModuleType()).Elem()
c.modules[providedType] = module
return
}
// 자동 타입 추론
c.modules[moduleType] = module
// 구현하는 인터페이스들 등록
for i := 0; i < moduleType.NumMethod(); i++ {
method := moduleType.Method(i)
// 인터페이스 매칭 로직
```

```
c.registerInterfaceImplementation(module, method)
}
}
```

7.5 모듈 레지스트리

7.5.1 전역 레지스트리

```
var (
registry = make(map[string]func() any)
mutex sync.RWMutex
)

// 모듈 등록
func RegisterModule(name string, constructor func() any) {
mutex.Lock()
defer mutex.Unlock()
if _, exists := registry[name]; exists {
panic(fmt.Sprintf("module %s already registered", name))
}
registry[name] = constructor
}
```

```go
// 모듈 생성
func New(name string) any {
mutex.RLock()
defer mutex.RUnlock()
constructor, exists := registry[name]
if !exists {
panic(fmt.Sprintf("module %s not found", name))
}
return constructor()
}
```

7.5.2 모듈 등록 예시

```go
// 각 모듈의 init 함수에서 등록
func init() {
modules.RegisterModule("chainedhotstuff", func() any {
return chainedhotstuff.New()
})
modules.RegisterModule("fasthotstuff", func() any {
return fasthotstuff.New()
})
modules.RegisterModule("bls12", func() any {
return bls12.New()
```

```
})

}
```

7.6 순환 의존성 해결

7.6.1 문제 상황

```
// A가 B에 의존

type ModuleA struct {

b ModuleB

}


// B가 A에 의존

type ModuleB struct {

a ModuleA

}
```

7.6.2 지연 초기화 해결책

```
// 1단계: 모든 모듈 생성 (의존성 없이)

a := &ModuleA{}

b := &ModuleB{}
```

```
// 2단계: Core에 등록
core.registerModule(a)
core.registerModule(b)

// 3단계: 초기화 (의존성 주입)
a.InitModule(core) // a.b = core에서 가져옴
b.InitModule(core) // b.a = core에서 가져옴
```

7.6.3 실제 구현

```
func (c *Core) initModules() {
// 모든 모듈에 대해 InitModule 호출
for _, module := range c.modules {
if initializable, ok := module.(Module); ok {
initializable.InitModule(c)
}
}
}
```

7.7 설정 시스템

7.7.1 Options 구조체

```
type Options struct {
```

```
id hotstuff.ID
eventLoopBufferSize int
shouldUseAggQC bool
shouldUseTree bool
connectionTimeout time.Duration
// ... 기타 설정들
}

type Option func(*Options)
```

7.7.2 설정 옵션들

```
func WithID(id hotstuff.ID) Option {
return func(opts *Options) {
opts.id = id
}
}

func WithEventLoopBufferSize(size int) Option {
return func(opts *Options) {
opts.eventLoopBufferSize = size
}
}
```

```go
func  WithAggQC() Option {
return  func(opts *Options) {
opts.shouldUseAggQC = true
}
}
```

7.7.3 설정 사용

```go
builder := modules.NewBuilder()
builder.AddOption(
modules.WithID(1),
modules.WithEventLoopBufferSize(1000),
modules.WithAggQC(),
)
```

7.8 이벤트 시스템 통합

7.8.1 이벤트 루프 등록

```go
func (c *Core) initModules() {
// 이벤트 루프를 먼저 등록
c.modules[reflect.TypeOf((*eventloop.EventLoop)(nil)).Elem()] = c.eventLoop
```

```go
// 다른 모듈들 초기화
for _, module := range c.modules {
if initializable, ok := module.(Module); ok {
initializable.InitModule(c)
}
}
}
```

7.8.2 이벤트 핸들러 등록 예시

```go
func (s *Synchronizer) InitModule(mods *modules.Core) {
var eventLoop *eventloop.EventLoop
mods.Get(&eventLoop)
// 타임아웃 이벤트 핸들러
eventLoop.RegisterHandler(TimeoutEvent{}, func(event any) {
timeoutEvent := event.(TimeoutEvent)
if s.View() == timeoutEvent.View {
s.OnLocalTimeout()
}
})
// 기타 핸들러들...
}
```

7.9 모듈 확장 예시

7.9.1 새로운 합의 알고리즘 추가

```
// 1. Rules 인터페이스 구현
type MyConsensus struct {
modules.Implements[consensus.Rules]
blockChain modules.BlockChain
logger logging.Logger
}

// 2. 모듈 초기화
func (mc *MyConsensus) InitModule(mods *modules.Core) {
mods.Get(&mc.blockChain, &mc.logger)
}

// 3. 합의 규칙 구현
func (mc *MyConsensus) VoteRule(proposal hotstuff.ProposeMsg) bool {
// 사용자 정의 투표 로직
return true
}

func (mc *MyConsensus) CommitRule(block *hotstuff.Block) *hotstuff.Block {
```

```go
// 사용자 정의 커밋 로직
return nil
}

func (mc *MyConsensus) ChainLength() int {
return 3
}

// 4. 모듈 등록
func init() {
modules.RegisterModule("myconsensus", func() any {
return &MyConsensus{}
})
}
```

7.9.2 새로운 암호화 방식 추가

```go
type Mycrypto struct {
modules.Implements[modules.CryptoBase]
privKey MyPrivateKey
pubKeys map[hotstuff.ID]MyPublicKey
}
```

```go
func (mc *MyConsensus) Sign(data []byte) (hotstuff.QuorumSignature, error) {
// 사용자 정의 서명 로직
return  nil, nil
}

func (mc *MyConsensus) Verify(sig hotstuff.QuorumSignature, data []byte) bool {
// 사용자 정의 검증 로직
return true
}

func (mc *MyConsensus) Combine(sigs ...hotstuff.QuorumSignature) (hotstuff.Quoru
// 사용자 정의 집계 로직
return  nil, nil
}
```

7.10 테스트 지원

7.10.1 모의 모듈 (Mock Modules)

```go
type MockBlockChain struct {
blocks  map[hotstuff.Hash]*hotstuff.Block
}
```

```go
func (mbc *MockBlockChain) Store(block *hotstuff.Block) {
mbc.blocks[block.Hash()] = block
}

func (mbc *MockBlockChain) Get(hash hotstuff.Hash) (*hotstuff.Block, bool) {
block, exists := mbc.blocks[hash]
return  block, exists
}

// 테스트에서 사용
func  TestConsensus(t *testing.T) {
builder := modules.NewBuilder()
builder.Add(
&MockBlockChain{blocks: make(map[hotstuff.Hash]*hotstuff.Block)},
chainedhotstuff.New(),
// ... 기타 모듈들
)
core := builder.Build()
// 테스트 수행
}
```

7.10.2 모듈 격리 테스트

```go
func TestSynchronizerOnly(t *testing.T) {
// 필요한 모듈만 추가
builder := modules.NewBuilder()
builder.Add(
&MockCrypto{},
&MockConfiguration{},
synchronizer.New(synchronizer.NewFixedViewDuration(time.Second)),
)
core := builder.Build()
var sync modules.Synchronizer
core.Get(&sync)
// Synchronizer만 테스트
}
```

7.11 성능 최적화

7.11.1 모듈 캐싱

```go
type cachedCore struct {
*Core
cache map[reflect.Type]any
mutex sync.RWMutex
}
```

```go
func (cc *cachedCore) Get(ptrs ...any) {
cc.mutex.RLock()
// 캐시에서 먼저 확인
allCached := true
for _, ptr := range ptrs {
elemType := reflect.TypeOf(ptr).Elem()
if _, exists := cc.cache[elemType]; !exists {
allCached = false
break
}
}
if allCached {
for _, ptr := range ptrs {
elemType := reflect.TypeOf(ptr).Elem()
reflect.ValueOf(ptr).Elem().Set(reflect.ValueOf(cc.cache[elemType]))
}
cc.mutex.RUnlock()
return
}
cc.mutex.RUnlock()
// 캐시 미스 시 원본 호출
cc.Core.Get(ptrs...)
```

```go
// 결과 캐싱
cc.mutex.Lock()
for _, ptr := range ptrs {
elemType := reflect.TypeOf(ptr).Elem()
cc.cache[elemType] = reflect.ValueOf(ptr).Elem().Interface()
}
cc.mutex.Unlock()
}
```

7.11.2 지연 로딩

```go
type lazyModule struct {
constructor func() any
instance any
once sync.Once
}

func (lm *lazyModule) getInstance() any {
lm.once.Do(func() {
lm.instance = lm.constructor()
})
return lm.instance
```

```
}
```

7.12 디버깅 지원

7.12.1 모듈 의존성 그래프

```
func (c *Core) PrintDependencyGraph() {
fmt.Println("Module Dependency Graph:")
for  moduleType, module := range  c.modules {
fmt.Printf("%s:\n", moduleType.Name())
// 리플렉션으로 의존성 분석
c.analyzeDependencies(module)
}
}
```

7.12.2 모듈 상태 모니터링

```
type ModuleMonitor struct {
modules  map[string]ModuleStatus
mutex sync.RWMutex
}

type ModuleStatus struct {
Name  string
```

```
Initialized  bool

LastActivity time.Time

ErrorCount  int

}



func (mm *ModuleMonitor) UpdateStatus(name  string, status ModuleStatus) {

mm.mutex.Lock()

defer  mm.mutex.Unlock()

mm.modules[name] = status

}
```

이러한 모듈 시스템을 통해 HotStuff는 복잡한 의존성 관계를 가진 컴포넌트들을 효율적으로 관리하고, 새로운 기능을 쉽게 추가할 수 있는 확장 가능한 아키텍처를 제공합니다.